

SECTION 21 — CI/CD FOR SENIOR SDET (Refined)

Topics Covered

- 21.1-21.42 (42 headers: fundamentals through ELK/Grafana)

First pass — 4 parallel batch subagents

21.1 CI/CD Fundamentals — Refined

Theory: CI = developers merge frequently to main, each merge triggers automated build + tests. Continuous Delivery = every commit yields production-deployable artifact with manual prod approval. Continuous Deployment = every green main auto-releases to prod, no human gate. Pillars: trunk-based development (<48h branches), automated tests, immutable versioned artifacts (never rebuild per env), fast-failing builds (<10min PR feedback), observability, rollback strategy. Enterprise Relevance (5000+ tests, Deloitte client delivery): Pipeline is the quality gate, not a cron job. SDET owns test stages, parallelization/sharding, retries, reporting, flake quarantine, and promotion blocks. Without CI discipline, 5000 tests rot into red-noise ignored by devs; with it, commit-to-prod lead time (DORA) drops from days to hours and escaped defects fall 60%+.

```
# .github/workflows/ci.yml — trigger every push/PR, fail fast
on: [push, pull_request]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', cache: 'maven' }
      - run: mvn -B verify
```

High-Stakes Scenario: Main red for 3 days, 200 devs blocked, client escalation. Triage: 1) `git bisect` + last-green artifact tag, 2) quarantine flaky suite to non-blocking (not delete), 3) fail-fast smoke first (2min) before full regression, 4) branch protection requires green checks, 5) postmortem: missing test isolation + no owned triage rotation. Restore in hours, not days. Anti-Patterns: Nightly-only runs (feedback 24h late); rebuilding artifact per env (untested binary in prod); long-lived feature branches (merge debt); red builds normalized ("rerun until green"); secrets in YAML. Qs: CI vs Delivery vs Deployment? Why immutable artifact? What makes a build fast-failing?

21.2 Pipeline Stages — Refined

Theory: Canonical flow: Source (push/PR/tag/cron) → Build (compile, deps, lint, package) → Unit Test (fast, mocked) → Static Analysis/SAST/SCA → Artifact Publish (immutable SHA tag, never `latest`) → Deploy to Test (migrations, health) → API/UI Tests (contract, smoke, regression shards) → Promote Staging/Prod (approvals, canary). Each stage gates the next; failure stops promotion and notifies owners via Slack/email + PR checks. Declarative Jenkins/GitHub/GitLab YAML is versioned pipeline-as-code for audit traceability. Enterprise Relevance: SDET owns test stages end-to-end. Keep stages fast

(<5min PR smoke), deterministic (pinned browsers/deps/Docker), observable (JUnit/Allure per stage). Deloitte audit needs stage-level evidence: what ran, what passed, what artifact promoted.

```
// Jenkinsfile declarative – stages as code
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'mvn -B package -DskipTests' } }
    stage('Unit Test') { steps { sh 'mvn -B test' } post { always { junit
'target/surefire-reports/*.xml' } } }
    stage('API Tests') { steps { sh 'mvn -B verify -Dgroups=api' } }
  }
}
```

High-Stakes Scenario: UI stage takes 45min blocking releases. Triage: split lint/unit (2min, parallel DAG) → API contract (5min) → E2E sharded (10min) → perf smoke; cache `~/m2/node_modules` by lockfile hash; fail-fast smoke before full; PR feedback 45→9min. Anti-Patterns: Monolithic single-job pipeline (no parallelism); test stage without JUnit publish (invisible failures); deploying untested artifact (skipped stages); snowflake agents (unreproducible). Qs: Draw 7 stages from memory? Where do SDET-owned stages sit? What artifact is promoted?

21.3 Build vs Test vs Deploy — Refined

Theory: Build = compile + resolve deps + lint + package immutable artifact with version/SHA tag. Test = validate artifact (unit, contract, API, UI, perf, security) against quality gates without mutating prod; never rebuilds, only exercises. Deploy = install a previously promoted artifact to an environment with migrations, config injection, health checks, traffic shift. Golden rule: build once, promote same binary Dev→QA→Staging→Prod; rebuilding per env tests a different binary than ships. Enterprise Relevance: SDET fails fast in build (compile/lint/unit <3min), comprehensively in test (smoke 5min → regression shards), verifies in deploy (post-deploy smoke + health). Root-cause analysis is impossible if QA tested build #101 but prod runs build #102.

```
mvn -B clean package -DskipTests          # build: artifact target/app-1.4.2.jar
mvn -B verify -Dgroups=regression         # test: exercises SAME jar
kubectl set image deploy/app app=registry/app:1.4.2 # deploy: promotes SAME tag
```

High-Stakes Scenario: Prod incident, QA says "passed in staging". Triage: discover staging tested rebuilt artifact (timestamp differs, dependency drift). Fix: immutable SHA tags, `latest` ban, promotion pipeline (not rebuild), SBOM attestation, artifact fingerprint in test report. Anti-Patterns: `mvn package` inside every env job; `latest` tag overwrite; testing against localhost build while deploying registry image; deploy without health/smoke gate. Qs: Why is rebuilding per env a defect? What proves QA and prod ran identical bytes?

21.4 Smoke Tests — Refined

Theory: Narrow, fast go/no-go suite verifying critical paths: app starts, login works, core APIs 200, DB reachable. Runs post-deploy on every build, blocks broken builds before expensive regression. Budget 5-10min, stable, environment-aware, fully automated (Playwright/RestAssured). Smoke failure stops promotion and triggers rollback; never rerun full regression on red smoke. Enterprise Relevance: At

Deloitte scale, smoke is the merge/deploy gate every PR and post-deploy check. Lean coverage (<20 cases) with clear assertions maximizes pipeline trust and dev feedback speed. Flaky smoke is worse than no smoke — it trains teams to ignore red.

```
// smoke.spec.ts — post-deploy gate, must pass in <10min
test('app healthy', async ({ request, page }) => {
  await expect((await request.get('/health')).ok()).toBeTruthy();
  await page.goto('/login');
  await page.getByLabel('Username').fill(process.env.SMOKE_USER!);
  await expect(page).toHaveURL(/dashboard/);
});
```

High-Stakes Scenario: Regression runs 2h on a build whose login is broken — 2h wasted × 10 shards. Triage: smoke (login + health + 1 checkout ping) runs first in DAG; on smoke fail, cancel downstream shards (cancel-in-progress), notify author in 3min, block merge. Track smoke pass rate as release-readiness signal. Anti-Patterns: 200-case "smoke" taking 40min; deep assertions in smoke (belongs in regression); environment-hardcoded URLs; smoke depending on test data created by other suites; ignoring smoke red ("will pass on retry"). Qs: Smoke vs sanity vs regression? What belongs in smoke and what never does? How long may smoke take?

21.5 Regression Tests — Refined

Theory: Regression validates new changes did not break existing functionality across integrated modules. Tiered in CI: PR-level targeted tests (impact-mapped), nightly full suite, pre-release risk-based slice. Tag by priority/owner/runtime (@regression @P1); parallelize with sharding; quarantine flaky; require Jira traceability, triage SLAs, trend dashboards. Selective automation + data-reset strategies + stable baselines balance thorough coverage with pipeline speed. Enterprise Relevance: Full regression nightly (not per-PR) keeps PR feedback <10min while catching integration breaks within 24h. Deloitte sign-off needs traceable pass evidence per release, not "all green locally".

```
# PR: targeted critical only; nightly: full
mvn -B test -Dgroups="regression-critical" -Dparallel=methods -DthreadCount=4
npx playwright test --shard=1/4 --retries=2 # nightly full, sharded
pytest -m "regression and not slow" -n 4
```

High-Stakes Scenario: 2000-test suite blocks every PR for 90min; devs bypass checks. Triage: split smoke (PR gate, 8min) from regression (merge/nightly); impact analysis (only tests touching changed services); shard 4×; quarantine top-20 flakes with owner+SLA. PR feedback 90→9min, nightly catches rest. Anti-Patterns: Full regression on every push; untagged monolith (cannot slice); no quarantine (flakes block all); missing traceability (cannot prove coverage to client). Qs: Tiered regression design? How do you slice PR vs nightly vs pre-release?

21.6 API Tests in Pipeline — Refined

Theory: API tests validate contracts, status codes, schemas, business logic, and integrations — faster and stabler than UI. Run post-deploy against ephemeral/test environments (RestAssured/Supertest/Newman) with env-injected base URLs. Include Pact/OpenAPI contract validation, data seeding/cleanup, parallel execution. Gate promotion on 100% critical API pass; log request/response on failure for rapid debugging (Jenkins/Actions reports, Allure). Enterprise Relevance:

API stage is the highest-value gate per minute: 500 contract checks in 3min catch backend breaks that 2h UI suites find late. Deloitte pipelines run API on every PR, UI smoke only, full UI nightly.

```
given().baseUrl(System.getenv("API_URL"))
  .when().get("/orders/{id}", 123)
  .then().statusCode(200).body("status", equalTo("SHIPPED"));
```

High-Stakes Scenario: UI suite green but prod orders fail — contract drift (new required field) undetected. Triage: add OpenAPI diff + Pact provider verification as merge gate; API suite covers all endpoints × happy/negative; UI asserts only journeys. Contract break blocks merge in 4min instead of escaping. Anti-Patterns: UI-only pipeline (slow, flaky); API tests with hardcoded baseUrl/tokens; no schema validation (field renames escape); logging full PII payloads. Qs: Why API before UI in pipeline order? What gates promotion — API or UI?

21.7 UI Tests in Pipeline — Refined

Theory: UI tests validate critical end-to-end journeys in real browsers — slower and flakier by nature. Strategy: lean smoke slice per commit (login + 3 journeys, <10min), full suite nightly. Dockerized browsers, headless, explicit waits, Page Objects, sharding. Isolate test data, capture videos/traces on failure, auto-retry once. Gate only on stable critical paths; never block deployment on known-flaky tests. Publish HTML reports to pipeline artifacts for audit/debugging. Enterprise Relevance: UI stage is the most expensive gate; misused it blocks all releases. Deloitte rule: smoke per PR, full nightly, quarantine list reviewed weekly. One flaky UI test blocking 200 devs costs more than the bug it guards.

```
- name: UI smoke (PR gate, 8min)
  run: npx playwright test --project=chromium --shard=1/4 --retries=1
- name: UI full (nightly, non-blocking)
  if: github.event.schedule == '0 1 * * *'
  run: npx playwright test --retries=2
```

High-Stakes Scenario: UI gate red 40% of PRs, all flakes (animation timing, third-party ads). Triage: quarantine 15 worst to nightly with tickets; stub ads/analytics via route abort; web-first assertions; smoke = 12 stable tests only. PR green rate 60→96% in 2 weeks. Anti-Patterns: Full UI on every push; headed browsers in CI (needs display); fixed sleeps; shared test users (parallel collisions); blocking prod on quarantine-list tests. Qs: What UI subset gates PR vs nightly? How do you stop UI stage from blocking all delivery?

21.8 Unit vs API vs UI Gates — Refined

Theory: Test pyramid enforced as pipeline gates. Unit: many, fast (<5min), mocked, >80% coverage — blocks merge. API/integration: moderate count, post-deploy to test env, contract + business logic — blocks environment promotion. UI E2E: few, slow, critical journeys only — smoke blocks, full suite advisory/nightly. Order by speed (fail-fast): lint → unit → API → UI smoke → full UI. Time budgets + quality thresholds (SonarQube, coverage) per gate. Optimizes feedback speed, cost, stability, defect localization. Enterprise Relevance: Wrong gate order (UI first) wastes 30min before a unit failure surfaces. Deloitte pipelines encode pyramid: unit gate on PR (minutes), API gate on deploy-to-QA, UI smoke gate on staging promotion, full UI informational.

```
gates:
  unit: { time: "<5min", coverage: ">80%", blocks: merge }
```

```

    api: { time: "<15min", blocks: promotion-to-staging, require: "100% critical pass"
  }
  ui: { smoke_only_pr: true, full_nightly: true, blocks: "smoke only" }

```

High-Stakes Scenario: Team runs UI before unit; a null-pointer (caught by unit in 20s) surfaces after 25min UI run, 10×/day. Triage: reorder DAG (lint+unit first, cancel downstream on fail), publish per-gate timings, track time-to-first-signal (target <4min). Anti-Patterns: Same gate for all layers (UI failures treated like unit); coverage gate on UI (meaningless); no time budget (gates drift to 60min); API failures marked advisory. Qs: Draw pyramid with gate ownership? Which gate blocks merge vs promotion vs nothing?

21.9 Environment Promotion — Refined

Theory: Promotion moves the same immutable artifact Dev→QA→Staging/UAT→Prod after passing gates, approvals, and compliance checks; each environment increases fidelity and data realism. Strategies: blue-green (two envs, instant switch, costly), canary (5%→25%→100% with analysis + auto-rollback, Argo Rollouts/Flagger). Automated smoke post-promotion; manual approval for prod. Environment parity via IaC + versioned configs. SDET verifies deployment health, migration scripts, feature flags per stage; rollback to prior artifact immediately if post-deploy metrics/tests fail. Enterprise Relevance: Promotion (not rebuild) is what makes staging results valid for prod. Deloitte regulated clients require promotion audit trail: artifact SHA, gates passed, approver, smoke result.

```

promote --artifact app:1.4.2 --from qa --to staging
kubectl argo rollouts promote app # canary approve after analysis
kubectl rollout undo deployment/api # rollback to last-known-good

```

High-Stakes Scenario: Staging green, prod red — configs drifted (staging flag on, prod off). Triage: version configs with artifact (Helm values per env in Git), diff env configs in pipeline, post-promotion smoke per env reading actual flag state, automated rollback on smoke fail. Anti-Patterns: Rebuilding for prod; manual `kubectl apply` from laptop; no post-promotion smoke; canary without metrics/rollback; promoting on UI-only green while API gate red. Qs: Blue-green vs canary vs rolling — when each? What proves staging-pass predicts prod-pass?

21.10 Configuration Management — Refined

Theory: Externalize environment-specific values from code per 12-Factor App: URLs, timeouts, feature flags, resource limits via env vars, ConfigMaps, or Spring profiles. Version configs in Git, template with Helm/Kustomize, validate per environment in pipeline. SDET parameterizes base URLs, browsers, test datasets without code changes. Prevent drift with IaC, config audits, pipeline injection checks. Ensures reproducible deployments and safe promotion of identical artifacts. Enterprise Relevance: Hardcoded URLs are the #1 cause of "works in QA, fails in staging". Deloitte pattern: same test code, different injected config per env; config diff is a pipeline artifact.

```

# k8s ConfigMap injected as env (no rebuild per env)
env:
  - name: API_BASE_URL
    valueFrom: { configMapKeyRef: { name: app-cfg, key: url } }
# test reads it:
base_url = os.environ["API_BASE_URL"]

```

High-Stakes Scenario: Staging tests hit prod DB (config copy-paste). Triage: per-env ConfigMaps + pipeline validation (prod URL pattern rejected in non-prod jobs), secrets/config separation, startup log of active env + target host (masked secrets) for every run. Anti-Patterns: URLs in test code; `if env==qa` branches in tests; unversioned manual config edits; config and secrets in same file. Qs: Where do SDET configs live vs app configs? How do you prove which config a run used?

21.11 Secrets Management — Refined

Theory: Never hardcode passwords, tokens, or keys in code, logs, or pipeline YAML. Store in Vault, AWS Secrets Manager, Azure Key Vault, or Jenkins/GitHub encrypted secrets with least-privilege IAM and rotation. Inject at runtime as masked env vars; scan repos with Gitleaks/TruffleHog; audit access. SDET uses test service accounts with scoped permissions and ephemeral tokens. Enforce short TTLs, automatic rotation, pipeline masking for SOC2/client compliance. Enterprise Relevance: A committed prod token in a public-adjacent repo is a breach + failed audit. Deloitte engagements require: no secrets in Git history (scan in PR gate), masked CI logs, per-env service accounts, rotation runbooks.

```
- uses: hashicorp/vault-action@v2
  with: { url: https://vault.corp, secrets: "kv/app apiKey | API_KEY" }
- run: npx playwright test
  env: { API_KEY: ${ secrets.API_KEY } } # masked in logs
```

High-Stakes Scenario: Secret leaked in CI log + committed `.env` (traces/videos capture typed secrets too). Triage: 1) rotate immediately, 2) purge history (BFG) + revoke, 3) Gitleaks gate on PR, 4) switch to API-login + storageState (credential appears once, not per test), 5) never `console.log(process.env)`. Anti-Patterns: `.env` committed; tokens in test code/screenshots/traces; shared admin creds across workers; long-lived tokens; secrets in Allure reports. Qs: Where do test secrets live at each layer (local/CI/prod)? What do you do in the first 10min after a leak?

21.12 Environment Variables — Refined

Theory: Env vars externalize config from code for CI/CD portability across dev/QA/prod (12-Factor). Scoped plain vars for URLs/browsers/tags; masked secrets for credentials. Local: `.env` (git-ignored) + `.env.example` committed; CI: vault/Jenkins credentials/GitHub Encrypted Secrets. Access via `process.env.BASE_URL` / `System.getenv`; validate required keys at startup, fail fast with clear message. Enterprise Relevance: Same suite runs everywhere by swapping env, not code. Deloitte pattern: `TEST_ENV=qa|staging|prod` + per-env files, CI real vars override files, active env logged per run (secrets masked).

```
env:
  BASE_URL: ${ secrets.QA_URL } # masked
  BROWSER: chromium
```

```
// fail fast, never default silently to prod
const baseUrl = process.env.BASE_URL;
if (!baseUrl) throw new Error('BASE_URL missing - set .env or CI secret');
```

High-Stakes Scenario: Staging run hits prod (missing env defaulted to prod URL). Triage: no silent defaults for URLs; per-env allowlist validation (prod pattern rejected outside prod job); log `ENV` +

target host every run; separate secrets per env. Anti-Patterns: Hardcoded URLs; committed `.env` ; silent prod default; secrets in plain vars visible in logs; env-specific `if` branches in tests. Qs: Plain vars vs secrets — what goes where? How do you fail fast on missing config?

21.13 Artifact Management — Refined

Theory: Runners are ephemeral — artifacts persist build outputs beyond the job for audit and debugging: JARs/zips, JUnit XML, logs, screenshots/traces/videos, coverage. Jenkins `archiveArtifacts` (fingerprint for traceability) or Actions `upload-artifact` ; promote binaries to Nexus/Artifactory/S3 with versioning + retention policies (e.g., 30d PR artifacts, 1yr release). Essential for Deloitte traceability, reruns, and client audit. Enterprise Relevance: "CI passed" without artifacts is unverifiable. Every release gate needs: test report + trace/video on failure + SBOM + artifact SHA, retained per policy.

```
archiveArtifacts artifacts: 'target/*.jar, playwright-report/**', fingerprint: true
```

```
- uses: actions/upload-artifact@v4
  with: { name: report-${{ matrix.shard }}, path: playwright-report, retention-days:
30 }
```

High-Stakes Scenario: Release disputed — client asks "what exactly was tested in build 412?" Triage without artifacts = guesswork. Fix: fingerprinted artifacts + merged reports + retention policy + artifact-to-commit traceability (SHA in report header). Anti-Patterns: No artifact upload (logs lost on runner kill); `latest` overwrite (history destroyed); infinite retention (cost explosion); test videos retained for passes (storage 10x). Qs: What artifacts must every pipeline keep, and for how long? How do you trace artifact → commit → report?

21.14 Test Reports — Refined

Theory: Reports convert raw XML/JSON results into actionable dashboards: pass rate, duration, history, flake trends. Publish JUnit/TestNG/Cucumber JSON or Allure to Jenkins JUnit plugin, GitHub Job Summaries/Pages. Gate releases on thresholds (fail build on breach). Deloitte needs: per-run report linked to commit + artifact SHA + environment, retained per policy. Enterprise Relevance: Developers ignore pipelines without readable reports. A failing build with no report link = no fix. Reports are the release-go/no-go evidence for managers.

```
- uses: dorny/test-reporter@v1
  if: always()
  with: { name: JUnit, path: '**/TEST-*.xml', reporter: java-junit, fail-on-error:
true }
- uses: actions/upload-artifact@v4
  if: always()
  with: { name: allure-results, path: allure-results }
```

High-Stakes Scenario: Nightly red, nobody knows which 30 of 2000 failed or why. Triage: Allure history + failure taxonomy (assert vs env vs flake), slowest-tests list, flake-rate trend, owner tags. Morning triage: reproduce top-3, bisect to commit, quarantine flakes with tickets. Anti-Patterns: Console-log-only results; report overwritten per shard (no merge); no history (cannot spot new failures); pass-rate

vanity without flake split. Qs: What does a manager need in a report to approve release? How do you merge sharded reports?

21.15 Screenshots/Videos/Traces as Artifacts — Refined

Theory: Headless CI failures are undebuggable without visual evidence. Capture: screenshots on failure, video on retry/failure, full traces (DOM snapshots + network + console + actions). Playwright: `screenshot: only-on-failure`, `video: retain-on-failure`, `trace: on-first-retry`. Upload `playwright-report/` + `test-results/` with 7-30d retention; replay locally via Trace Viewer (`npmx playwright show-trace`). Enterprise Relevance: Visual artifacts cut MTTR from hours (repro locally) to minutes (watch trace). Deloitte audit: every failed release-gate test must link trace/video. Storage cost controlled by failure-only retention.

```
// playwright.config.ts – evidence without storage explosion
use: { screenshot: 'only-on-failure', video: 'retain-on-failure', trace: 'on-first-retry' }
```

```
- uses: actions/upload-artifact@v4
  if: failure()
  with: { name: trace-report, path: playwright-report, retention-days: 14 }
```

High-Stakes Scenario: CI-only failure, passes locally, no evidence — 3 devs spend a day guessing. Triage: trace shows API 500 after deploy (backend, not test); video shows modal overlay; console shows CORS. Fix in 20min with artifacts vs day without. Anti-Patterns: No artifacts (blind); `trace: on` always (storage 10x, slow); videos of passing tests retained; traces containing secrets committed. Qs: Which artifact answers what (screenshot vs video vs trace)? What retention balances cost vs audit?

21.16 Parallel Jobs — Refined

Theory: Parallel jobs cut pipeline time by running independent suites concurrently across agents. Jenkins: `parallel` stages in one pipeline. GitHub Actions: multiple `jobs` (parallel by default) with `needs` for sequencing. Requires test independence, isolated test data, thread-safe fixtures — shared state causes collisions and shared-state flakes that vanish serially. Enterprise Relevance: Lint + unit + API-contract in parallel DAG turns 18min serial into 6min. Deloitte rule: fastest checks first, independent jobs parallel, dependent (deploy→smoke) sequential via `needs`.

```
// Jenkins – parallel stages, fail fast
stage('Test') {
  parallel {
    stage('Unit') { steps { sh 'mvn -B test' } }
    stage('API') { steps { sh 'mvn -B verify -Dgroups=api' } }
  }
}
```

```
# Actions – jobs run parallel; needs sequences
jobs:
  unit: { runs-on: ubuntu-latest, steps: [{ run: mvn -B test }] }
  api: { runs-on: ubuntu-latest, steps: [{ run: mvn -B verify -Dgroups=api }] }
```



```
report: { needs: [unit, api], runs-on: ubuntu-latest, steps: [{ run: ./merge.sh }]
}
```

High-Stakes Scenario: Parallel jobs share one test DB → unique-constraint failures only in CI. Triage: per-job schemas (`testdb_job_${GITHUB_JOB}`), or serialize DB-touching jobs, or Testcontainers per job. Independence verified by running jobs in isolation vs together. Anti-Patterns: Parallel jobs sharing files/DB/users; `needs` missing (report merges before tests finish); unbounded parallelism hitting API rate limits. Qs: Jobs vs steps parallelism? How do you prove jobs are truly independent?

21.17 Matrix Builds — Refined

Theory: Matrix fans out one job definition across axes (OS × browser × Java/Node) to validate compatibility without YAML duplication. GitHub `strategy.matrix`, Jenkins Declarative Matrix `axis`, GitLab `parallel:matrix`. Catches environment-specific bugs early (Windows path separators, Safari rendering, Java 17 vs 21). Limit combinations (cost explodes: 3 OS × 3 browsers × 2 Java = 18 jobs); use `exclude / include` to prune, `fail-fast: false` for full diagnostics. Enterprise Relevance: Deloitte clients run Windows + macOS + Linux across Chrome/Edge/Firefox. Matrix proves portability per PR; without it, "works on my Ubuntu" ships broken to client Windows.

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, windows-latest]
    browser: [chromium, firefox]
    exclude: [{ os: windows-latest, browser: firefox }] # prune unsupported
  steps: [{ run: npx playwright test --project=${{ matrix.browser }} }]
```

High-Stakes Scenario: Release breaks only on Windows (path `join` vs `resolve`) + Safari (date parsing). Triage: matrix caught both pre-merge; without matrix, client UAT finds them 3 weeks later at 10x fix cost. Track per-axis pass rates to spot systemic env issues. Anti-Patterns: Full cross-product without pruning (18 jobs × 10min = waste); matrix without artifact separation (reports overwrite); `fail-fast: true` hiding other env failures. Qs: How do you bound matrix cost? What belongs in matrix vs nightly?

21.18 Test Sharding — Refined

Theory: Sharding splits a large suite into balanced chunks run on separate runners, then merges reports. Playwright `--shard=k/n`, Jest `--shard`, Maven Surefire forks. Balance by historical timing data, not file count (one 8min file vs ten 30s files unbalances shards). Dramatically cuts PR feedback (60→10min). Requires blob-per-shard + merge job (`merge-reports`), independent tests, and shared-nothing data. Enterprise Relevance: The standard answer to "suite too slow". Deloitte pattern: GitHub matrix `shard: [1/4, 2/4, 3/4, 4/4]` + merge job with `needs: [test]`; duration-balanced via `--reporter=blob` timing.

```
strategy: { fail-fast: false, matrix: { shard: [1, 2, 3, 4] } }
steps:
  - run: npx playwright test --shard=${{ matrix.shard }}/4 --reporter=blob
  - uses: actions/upload-artifact@v4
    with: { name: blob-${{ matrix.shard }}, path: blob-report }
merge:
```

```
needs: [test]
steps: [{ run: npx playwright merge-reports --reporter html ./all-blob-reports }]
```

High-Stakes Scenario: Shard 4/4 takes 22min, others 6min (one giant file). Triage: split giant file, use timing-based balancing (`--shard` with `fullyParallel` per-test), monitor slowest-shardDefines-total-time. Never add machines before balancing. Anti-Patterns: File-count balancing; shared DB across shards (collisions); no merge job (4 separate unreadable reports); sharding order-dependent tests (different shards = different failures). Qs: Sharding vs workers (when each)? How do you balance shards? What defines total time?

21.19 Worker Scaling — Refined

Theory: Worker scaling matches executor capacity to demand. Jenkins: static agents, Kubernetes pods (ephemeral, auto-scaled), EC2 fleets. GitHub: larger runners (`ubuntu-latest-8-cores`) or autoscaled self-hosted ARC on AKS/EKS. Monitor queue time, CPU, concurrency limits. Prefer ephemeral workers (clean state, security — no leftover files/secrets between jobs). Enterprise Relevance: Queue time is invisible pipeline tax: 15min queue + 10min tests = 25min feedback blamed on "slow tests". Deloitte SRE view: track queue-time P95 separately from test time; scale workers when queue >2min sustained.

```
runs-on: ubuntu-latest-8-cores # CPU-bound suites
# ARC autoscale (self-hosted, burst):
# minRunners: 2, maxRunners: 20, scale on queue length
```

High-Stakes Scenario: Monday 9am, 50 PRs queue on 2 runners — 40min waits, devs bypass CI. Triage: ARC max 2→20 on queue depth; split heavy UI to nightly; cache deps (queue + cold install compounds); ephemeral runners prevent "works on runner-3" state leaks. Anti-Patterns: Static 2 runners for 100 devs; persistent self-hosted runners with leftover state/secrets; scaling workers before sharding/caching (pays for waste). Qs: Queue time vs test time — which dominates your pipeline? Ephemeral vs persistent runners trade-off?

21.20 Retry Policies — Refined

Theory: Retry policies automatically rerun transient failures (network timeouts, 503s, browser launch flakes) without manual reruns. Jenkins Naginator (`retry: 2`), Actions `nick-fields/retry` , Playwright `retries: 2` (CI only, 0 locally to expose bugs fast). Cap attempts (1-2), add backoff/jitter, log attempts distinctly. Retries are a diagnostic buffer, not a fix — every retry must file or link a flake ticket. Enterprise Relevance: Unbounded retries hide real regressions and inflate CI 3x. Deloitte policy: `retries=2` CI, quarantine list for persistently flaky, flake-rate dashboard; a test passing only on retry 3 is a failing test.

```
// playwright.config.ts – retries on CI only
export default defineConfig({ retries: process.env.CI ? 2 : 0 });
```

```
// Jenkins – retry whole stage once on infra error
retry(1) { sh 'npx playwright test' }
```

High-Stakes Scenario: Suite green via 5 retries each — real checkout bug hidden for 2 weeks, escapes to prod. Triage: cap retries at 2, mark pass-on-retry as `flaky` (not passed), alert on flake-rate >2%,

root-fix waits/data/isolation, never raise retries to "stabilize". Anti-Patterns: retries=5+; retries locally (hides bugs during dev); retrying assertion failures (deterministic — fail fast); no retry logging (cannot distinguish flaky from stable). Qs: Retry vs quarantine — when each? How do you prevent retries from masking regressions?

21.21 Flaky Test Handling — Refined

Theory: Flaky tests (pass+fail identical code) erode trust until teams ignore red builds — then real regressions escape. Process: detect (rerun analytics, Allure history, Playwright HTML retries categorization passed/flaky/failed), quarantine (tag @flaky , separate non-blocking pipeline, auto-file Jira with trace + owner + SLA 7-14d), fix root (waits, test-data isolation, clock, order-dependence), reinstate only after N green runs. Block new flakes via gates (fail PR introducing flake). Enterprise Relevance: Flake rate is a release-readiness metric alongside pass rate. Deloitte rule: quarantine >5% suite = freeze features, hold flakiness hackathon; track flake-rate <1-2% healthy.

```
@Test(retryAnalyzer = RetryAnalyzer.class, groups = {"quarantine"})
public void flakyCheckoutTest() {} // non-blocking job, ticket FLAKY-123, owner +
SLA
```

High-Stakes Scenario: 30% suite flaky, nightly red 3 weeks, team ships "probably fine" — P1 escapes. Triage: quarantine worst 20 (pipeline green in a day), fix in priority order (waits → data → order → env), measure flake-rate weekly, gate new flakes. Green-in-week, stable-in-month. Anti-Patterns: Ignoring flakes ("rerun until green"); deleting flaky tests (coverage loss, no ticket); infinite retries; blaming "Selenium is flaky" without triage. Qs: Detect → quarantine → fix → reinstate workflow? What flake rate triggers feature freeze?

21.22 Fail-Fast vs Continue — Refined

Theory: Fail-fast stops the pipeline on first decisive failure to save time, compute, and developer wait: `strategy.fail-fast: true`, Jenkins `parallelsAlwaysFailFast()`, `pytest --maxfail=1`, `jest --bail`. Continue mode (`fail-fast: false` + `continue-on-error: true`) runs all checks for full diagnostics — ideal for cross-platform matrices where every env result matters. SDET pattern: order smoke/lint/unit first (fast, decisive), gate expensive integration behind them, cancel redundant shards on smoke fail, preserve independent failures for triage. Track time-to-first-signal vs hidden-second-defect cost. Enterprise Relevance: Fail-fast on PR (first red in 2min, cancel rest); continue on nightly/matrix (full failure inventory for triage). Wrong mode wastes either compute (continue on doomed build) or diagnostics (fail-fast hiding 5 more failures).

```
strategy: { fail-fast: true } # PR: stop at first red
# nightly matrix: full picture
strategy: { fail-fast: false }
pytest -x --maxfail=1 # fail fast locally
```

High-Stakes Scenario: Matrix of 12 envs, 1 fails fast and cancels other 11 — env-specific regression in Safari hidden for a week. Triage: PR jobs fail-fast (speed), nightly matrix continue (coverage); `continue-on-error` for advisory checks (audit) with separate required/optional status checks. Anti-Patterns: Always fail-fast (hides matrix failures); always continue (burns 40min on dead build); no `cancel-in-progress` for superseded PR pushes (queue clog). Qs: When fail-fast vs continue? How do required vs advisory checks interact?

21.23 Quality Gates — Refined

Theory: Quality gates answer "is this build releasable?" by enforcing thresholds on new code: SonarQube Sonar-way (zero new issues, reviewed hotspots, coverage $\geq 80\%$, duplication $\leq 3\%$, A ratings for reliability/security/maintainability). Jenkins/GitHub `sonar.qualitygate.wait=true` fails the job on status `!= OK` with auto-regress assignment. SDET practice: block merges on coverage, SAST, flake rate; risk-based GO/CAUTION/STOP rather than binary pass/fail; version gates per service. Enterprise Relevance: Gates are the contract between engineering and release management. Deloitte engagements require gate evidence per build: coverage delta, new vulnerabilities, flake rate — GO ships, CAUTION needs waiver + ticket, STOP blocks.

```
// Jenkins — block merge on gate
waitForQualityGate abortPipeline: true
```

```
- name: SonarQube Scan
  run: sonar-scanner -Dsonar.qualitygate.wait=true
```

High-Stakes Scenario: Coverage gate at 80% overall lets new untested payment module ship (80% average hides 0% new). Triage: gate on new-code coverage ($\geq 80\%$ on changed lines), not overall; separate gates for security hotspots (zero tolerance) vs coverage (waiver with ticket + expiry). Anti-Patterns: Overall-coverage gate (hides new debt); disabled gates "temporarily" (permanent); gate without waiver process (devs bypass); vanity 100% gate (wasted effort on generated code). Qs: New-code vs overall coverage — which gates? GO/CAUTION/STOP vs pass/fail?

21.24 Branch Strategies — Refined

Theory: GitFlow uses long-lived `main/develop/feature/release/hotfix` branches for scheduled versioned releases — stabilization windows and easy maintenance, but merge debt and slow CI feedback. Trunk-based development commits to `main` via $<48h$ branches behind feature flags, requiring strong automation and $<10min$ builds, enabling daily deploys (Google/Meta). Hybrids: GitHub Flow (PR-to-main), Release Flow (release branches from main), GitLab Flow (environment branches). SDET rule: protect `main` always-deployable with required status checks regardless of model. Enterprise Relevance: Branch model dictates CI design: GitFlow needs per-branch pipelines + release gates; trunk needs fast PR gates + flag cleanup. Deloitte client teams on GitFlow suffer week-long release branches with untested merges — SDET must gate each branch + nightly merge-to-main regression.

```
git checkout -b feature/xyz main # merge same day (trunk)
# GitFlow: feature → develop → release → main + hotfix
```

High-Stakes Scenario: Release branch lives 3 weeks, merges 40 features untested together — release day explodes. Triage: nightly regression on release branch from day 1, feature flags for incomplete work, branch protection (green checks required), merge-train/queue to serialize. Anti-Patterns: Long-lived branches without CI; direct pushes to main; flags without cleanup (tech debt); release branch tested only at release. Qs: Which model fits daily releases vs quarterly releases? What CI gates does each require?

21.25 Pull Request Validation — Refined

Theory: PR validation runs fast pre-merge checks on every push: build, lint, unit, SAST, dependency scan, selective tests via coverage mapping, plus title/body/branch conventions. GitHub required status checks, `gh pr checks --watch`, merge queues (bors/merge-train) serialize merges; targeted jobs (`npm test -- --grep "@unit|@smoke"`) block merge until green. Lightweight PR suite (~10-40min) with full suite at merge; rerun flaky checks via empty commit; small PRs linked to tickets. Enterprise Relevance: PR gate is the cheapest defect filter — a 10min gate catching 80% of breaks saves nightly/regression cycles. Deloitte rule: required checks (build, unit, smoke, SAST) block merge; advisory checks (perf, visual) warn.

```
on: { pull_request: { branches: [main] } }
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run lint && npm test -- --grep "@unit|@smoke"
      - run: npm audit --audit-level=high
```

High-Stakes Scenario: Flaky PR checks retried blindly; red main merged via admin override "just this once" — breaks 50 devs. Triage: branch protection (no bypass without 2 approvals + ticket), merge queue (rebase + retest before merge), flaky quarantine (don't block on known-flaky, but ticket), empty-commit rerun tracked (3 reruns = investigate). Anti-Patterns: Optional checks (ignored red); 60min PR suite (devs bypass); admin-override culture; giant PRs (unreviewable, untestable); no ticket linkage. Qs: Required vs advisory checks? How do merge queues prevent main breakage?

21.26 Scheduled Regression — Refined

Theory: Scheduled regression runs broader suites on cron rather than every commit: smoke on PR, selective regression on merge, full regression nightly `0 1 * * *` (GitHub `schedule`, GitLab trigger, Jenkins cron). Docker fresh staging stack, lockfile-keyed cache, baseline failure criteria (p95/error-rate), Slack/email only on failure. Stagger times, rotate baselines biweekly to avoid drift blindness. Enterprise Relevance: Cron suites catch integration drift that PR-scoped suites miss (cross-service changes landing separately). Deloitte pattern: PR (10min) → merge (30min selective) → nightly full (2h, non-blocking) → weekly perf/compat.

```
on:
  schedule: [{ cron: '0 1 * * *' }] # nightly 1am
  workflow_dispatch: {} # manual trigger
jobs:
  regression:
    runs-on: ubuntu-latest
    steps: [{ run: npx playwright test --retries=2 }]
```

High-Stakes Scenario: Cron suite red 2 weeks, nobody watches (alert fatigue, always-red). Triage: failures-only alerts (no green spam), morning triage rotation with SLA (acknowledge by 10am), quarantine-or-fix rule (no third red day without ticket), history dashboard distinguishing new vs known failures. Anti-Patterns: Cron without owner (red ignored); same suite as PR (no added value); alerting on success (mute fatigue); staging data drift (cron fails on data, not code). Qs: PR vs merge vs nightly vs weekly — what runs where? How do you keep cron suites trusted?

21.27 Nightly Suites — Refined

Theory: Nightly suites validate full-day integrations off-hours: comprehensive E2E, cross-browser, visual, performance, compatibility — too slow for PRs. Point at stable staging/QA URL (not ephemeral previews), dispatch as single suite run for readable history, alert failures-only (avoid mute fatigue), enforce morning triage: reproduce, bisect to commit, quarantine flaky. Stable `data-testid` selectors, dynamic waits, parallel shards, logs/reports for fast debug. Enterprise Relevance: Nightly is the integration safety net for cross-team changes. Deloitte pattern: nightly full + weekend perf/compat; triage rotation with 10am acknowledge SLA; flake quarantine before third red day.

```
npm test -- --grep "@cross-browser|@extended" --retries=2
# cron 1am, single run, history preserved per night
```

High-Stakes Scenario: Nightly red 3 weeks straight, 200 failures, nobody triages ("known flaky"). Triage: bulk-triage day (bucket by signature: 120 locator drift from design-system upgrade → 1 framework fix; 50 env; 30 real), fix-or-quarantine each bucket, restore trust in a week. Lesson from Sec 34: single component change → hundreds red, one root cause. Anti-Patterns: Nightly without owner/triage; ephemeral preview URL (data vanishes); alerting green (mute); same as PR suite (no added coverage); 6-month-old baselines (drift blindness). Qs: Nightly vs scheduled regression — difference? What makes nightly results trustworthy?

21.28 Deployment Validation — Refined

Theory: Deployment validation proves the deployed artifact works in its real environment, not just CI. After `skaffold apply` or blue-green/canary rollout: verify containers (exit-code 0 = pass), Playwright smoke against `SMOKE_BASE_URL` (not localhost), health checks, rollout status, DB connectivity, key read/write paths. Cloud Deploy, Argo, Azure verify tasks auto-fail rollout on error. Separate smoke config, upload report on failure, monitor short health window post-deploy. Enterprise Relevance: CI-green + deploy-broken is the classic escape (env-specific config, migration failure, traffic-shift issue). Deloitte gates require post-deploy smoke as promotion condition, not optional extra.

```
// smoke.config.ts — reads deployed env, not CI default
test.use({ baseURL: process.env.SMOKE_BASE_URL! });
test('post-deploy health', async ({ request }) => {
  await expect((await request.get('/health')).ok()).toBeTruthy();
});
```

```
kubectl rollout status deployment/api --timeout=120s
```

High-Stakes Scenario: Deploy succeeds, smoke skipped ("CI was green") — migration failed silently, prod checkout 500s for 2h. Triage: verify-task as rollout gate (auto-fail), smoke reads/writes (not just health ping), DB migration checksum check, 5min health window monitoring before traffic shift completes. Anti-Patterns: No post-deploy check; smoke against wrong env (CI URL); health-ping only (misses broken journeys); manual "looks good" approval without evidence. Qs: CI test vs post-deploy validation — what differs? What auto-fails a rollout?

21.29 Rollback Validation — Refined

Theory: Rollback validation ensures safe recovery to last-known-good when verification fails. Strategies: automatic rollback on failed smoke/tests/alerts (ECS circuit-breaker, OCI automatic rollback, `rollback-on-error` redeploy, blue-green traffic revert); manual `kubectl rollout undo` or single-stage redeploy; forward hotfix via flags. AWS upgrade-downgrade testing verifies backward compatibility: two-phase schema changes, state deserialization both ways. SDET practice: predefine failure criteria, test rollback drills, audit reason, verify data unaffected. Enterprise Relevance: Deploying without a tested rollback is a gamble. Deloitte release runbooks require: rollback decision owner, trigger metrics, max time-to-rollback (e.g., 10min), and data-migration reversibility proof before every prod deploy.

```
kubectl rollout status deployment/api --timeout=120s \
|| kubectl rollout undo deployment/api
# forward fix alternative:
kubectl set image deploy/api api=registry/api:1.4.1 # last-known-good
```

High-Stakes Scenario: Bad deploy corrupts data, rollback restores code but not DB (migration irreversible) — extended outage. Triage: expand-contract migrations (additive first, cleanup later), rollback drills in staging (measure time + data integrity), forward-fix vs rollback decision matrix pre-agreed, audit log of who rolled back and why. Anti-Patterns: No rollback plan ("roll forward only"); irreversible migrations; untested rollback (first use in prod emergency); rollback without data verification. Qs: Rollback vs roll-forward vs blue-green revert — when each? How do you test rollback before needing it?

21.30 Pipeline Performance Optimization — Refined

Theory: Optimize by measuring the critical path, then parallelizing, caching, and right-sizing. Split lint/unit/security into parallel DAG jobs with `needs`; shard tests (`parallelism: 4`) + timing split; cache `~/.npm/~/~/.m2` keyed by lockfile hash with pull-push policy; slim Docker layers + multi-stage builds + `npm ci`; incremental scans with full nightly scan; interruptible pipelines (cancel superseded); larger runners for CPU-bound jobs. Target PR feedback <5min. SDET practice: fastest checks first, artifacts reuse, fix flakes (flakes cost more than any optimization). Enterprise Relevance: Pipeline speed is developer productivity: 45min PR waits × 100 devs = 75 dev-hours/day burned. Optimization ROI is immediate and visible to leadership.

```
- uses: actions/setup-node@v4
  with: { node-version: 20, cache: 'npm' } # lockfile-keyed cache
jobs:
  lint: { runs-on: ubuntu-latest, steps: [{ run: npm run lint }] }
  unit: { runs-on: ubuntu-latest, steps: [{ run: npm test -- --grep @unit }] }
  e2e: { needs: [lint, unit], strategy: { matrix: { shard: [1, 2, 3, 4] } } }
```

High-Stakes Scenario: Pipeline 50min, team blames "tests slow" — profile shows 20min `npm install` (no cache) + 15min Docker build (no layer cache) + 15min serial tests. Triage: cache (20→2min), multi-stage + layer order (15→4min), sharding (15→5min). Total 50→11min without touching a single test. Anti-Patterns: Optimizing tests before caching/install (wrong bottleneck); `npm install` instead of `npm ci`; full scans on every PR; no `cancel-in-progress` (stale pushes burn agents). Qs: How do you find the critical path? Cache, parallelize, or right-size first — how do you decide?

21.31 CI Observability — Refined

Theory: CI observability combines monitoring (metrics/logs/traces) plus DORA signals: deployment frequency, lead time, change-fail rate, MTTR — visualized in Grafana/Datadog/CI Watch dashboards via OpenTelemetry. Track queue time, duration variance, flake rate, canceled minutes, per-job stability (healthy/flaky/broken), test history with stack traces. SDET practice: structured logs, trace spans across services, alert on degradation, avoid perfect-dashboard trap, correlate failures to commits, schedule DORA reports. Enterprise Relevance: Without observability, pipeline debates are opinion ("CI feels slow"). With it: queue P95 12min → scale runners (data wins budget); flake rate 8% → quarantine sprint; lead time 3d → trunk + flags initiative. DORA Elite targets guide investment.

```
# DORA deployment record (Datadog)
datadog-ci dora deployment --git-commit-sha "$SHA" --finished-at now
# Key dashboards: lead-time trend, change-fail %, MTTR, flake-rate by suite
```

High-Stakes Scenario: Leadership asks "is quality improving?" — no data, anecdotes only. Triage: DORA baseline (frequency, lead, fail rate, MTTR) + quality trends (escape rate, flake rate, coverage delta) in quarterly steering pack; every claim paired with chart + commit evidence. Anti-Patterns: Vanity dashboards (green % only); no DORA tracking; alerting on every failure (mute); metrics without owners/actions; perfect-dashboard trap (polishing charts vs fixing flakes). Qs: Four DORA metrics and what each reveals? What single chart proves pipeline health to a partner?

21.32 Jenkins — Refined

Theory: Jenkins orchestrates SDET CI via multibranch pipelines, agents (static/K8s/EC2), and stages: checkout → install → test → archive. Declarative `Jenkinsfile` syntax (preferred) with parallel stages for UI/API suites, JUnit/Allure publishing, failure retries. Credentials store + SCM webhooks for triggers. Deloitte expects pipeline-as-code, shared libraries (DRY across repos), and flaky-test quarantine knowledge. Enterprise Relevance: Jenkins remains dominant in enterprise clients (on-prem, custom agents, complex approvals). SDET must own `Jenkinsfile` : parallel UI/API, artifact archival, quality gates, Slack alerts. Shared libraries prevent 20 repos drifting.

```
stage('E2E') {
  steps { sh 'npx playwright test --reporter=junit' }
  post { always { junit 'results/*.xml'; archiveArtifacts 'playwright-report/**' } }
}
```

High-Stakes Scenario: Jenkins queue 30min (2 static agents, 50 jobs Monday). Triage: K8s ephemeral agents autoscaled on queue, parallel stages, cache deps, `timeout{}` on stuck builds, executor/agent-offline monitoring. Queue P95 30→3min. Anti-Patterns: Freestyle UI jobs (unversioned, unreviewable); no shared libs (copy-paste Jenkinsfiles); secrets in Groovy code; no agent labels (UI tests on wrong OS); infinite builds (no timeout). Qs: Declarative vs scripted — when each? How do shared libraries prevent drift?

21.33 GitHub Actions — Refined

Theory: GitHub Actions runs tests on `push`, `pull_request`, `schedule` via workflows in `.github/workflows/`. Jobs (parallel VMs) contain sequential steps sharing FS; needs sequences; `strategy.matrix` fans out browsers; setup actions (Node/Java) + caching + service containers (Selenium Grid, Postgres); artifact upload (traces/videos); branch protection requires green checks; Secrets for staging creds. Enterprise Relevance: Actions is the default for GitHub-hosted Deloitte

projects. SDET must write matrix + sharding + caching + artifact workflows that juniors can copy. PR feedback <10min determines adoption.

```
- run: npx playwright install --with-deps chromium
- run: npx playwright test --shard=${{ matrix.shard }}/4 --reporter=blob
- uses: actions/upload-artifact@v4
  if: failure()
  with: { name: report-${{ matrix.shard }}, path: blob-report }
```

High-Stakes Scenario: Actions minutes exploding (\$3k/mo) — full browsers × 3 OS × every push.

Triage: Chromium-only on PR, full matrix nightly; `--with-deps` cached Docker image; path filters (docs-only skips tests); larger runners only for UI shards; retention 14d not 90d. Anti-Patterns:

latest action versions (supply-chain breakage — pin SHAs); secrets in plain env; no `cancel-in-progress` (stale pushes burn minutes); macOS runners for Linux-testable suites (10x cost). Qs: Jobs vs steps vs matrix? How do service containers replace local Grid?

21.34 Azure DevOps — Awareness — Refined

Theory: Azure Pipelines uses `azure-pipelines.yml` with stages → jobs → pools → tasks (Maven, Node, Docker, PublishTestResults). Hosted Microsoft agents or self-hosted scale sets; variable groups + service connections + Key Vault for secrets; JUnit/Cobertura/Allure published to Tests tab; approvals + environments + parallel jobs; Boards work-item linking for traceability. Enterprise Relevance: Azure-shop clients (banks, government) standardize on DevOps + Boards + Test Plans. SDET awareness: publish results to Tests tab (not just logs), gate releases with environment approvals, link runs to Boards items for audit.

```
- task: PublishTestResults@2
  inputs: { testResultsFormat: JUnit, testResultsFiles: '**/TEST-*.xml' }
- task: DownloadPipelineArtifact@2
  inputs: { artifact: playwright-report }
```

High-Stakes Scenario: Pipeline green but Tests tab empty — failures invisible to managers, release approved blindly. Triage: PublishTestResults every job (even failed, `condition: always()`), environment approvals with test-evidence checklist, Boards linkage enforced by branch policy. Anti-Patterns: Logs-only results (no Tests tab); classic UI pipelines (unversioned); secrets in variable groups (use Key Vault); no environment approvals for prod. Qs: How do Tests tab + Boards + approvals form an audit chain? Hosted vs scale-set agents?

21.35 Docker — CI Awareness — Refined

Theory: Docker ensures reproducible CI by packaging browsers, drivers, JDK/Node, and tests into versioned images — eliminating "works on my machine". Lean Dockerfiles (layer caching, non-root users, `.dockerignore`), build image once, run lint/unit/integration as containers, push to ECR/ACR. Docker Compose for app+DB+Selenium Hub locally mirrors CI parity for debugging. Enterprise Relevance: Environment parity is the #1 CI flake killer. Deloitte pattern: Playwright's official image (`mcr.microsoft.com/playwright:v1.49-jammy`) as base; same image local + CI; Compose stack (app + Postgres + Hub) for pre-push verification.

```
FROM mcr.microsoft.com/playwright:v1.49-jammy
WORKDIR /app
```

```

COPY package*.json ./
RUN npm ci
COPY . .
RUN npx playwright test --list # validate install

```

```

# compose: app + db + hub parity
services:
  app: { build: ., ports: ["3000:3000"], depends_on: [db] }
  hub: { image: selenium/hub:4.41 }

```

High-Stakes Scenario: CI uses Ubuntu 24 + Node 22 while devs use Node 18 + macOS — native module failures only in CI. Triage: pin base image digest, `npm ci` (not `install`), non-root user (OpenShift requires), `.dockerignore` (`node_modules/.git` excluded), local `docker compose up` repro before pushing. Anti-Patterns: `latest` base (drifts); root user (OpenShift reject); `COPY .` before `npm ci` (cache bust every change); testing outside container while CI inside (parity gap). Qs: What goes in the test image vs mounted at runtime? How does Compose give pre-push confidence?

21.36 Selenium Grid in CI — Refined

Theory: Grid 4 enables parallel cross-browser execution in CI via Hub/Router plus Node containers (Chrome, Firefox, Edge). Deploy with Docker Compose or Kubernetes, scale nodes dynamically, route via `RemoteWebDriver`. Optimize with session queuing, video recording, retries, and TestNG suite sharding. Capture logs, VNC, timings for flaky analysis under load. Enterprise Relevance: Grid in CI replaces 10 sequential browser runs with 1 parallel matrix. Deloitte pattern: Compose Hub + 4 Chrome + 2 Firefox nodes as CI service; TestNG `thread-count` matched to slots; video on failure; queue metrics alert on saturation.

```

// SDET must know: RemoteWebDriver against CI Grid, quit frees slot
WebDriver driver = new RemoteWebDriver(
    new URL(System.getenv("GRID_URL")), new ChromeOptions());
try { /* tests */ } finally { driver.quit(); }

```

```

services:
  selenium-hub: { image: selenium/hub:4.41, ports: ["4444:4444"] }
  chrome: { image: selenium/node-chrome:4.41, shm_size: 2gb,
    environment: [SE_EVENT_BUS_HOST=selenium-hub, SE_NODE_MAX_SESSIONS=4] }

```

High-Stakes Scenario: Grid queue 20min — 2 nodes, 8 threads, video always-on filling disk. Triage: nodes 2→6, `max-sessions` 4, video retain-on-failure only, TestNG `thread-count` = slots, session-timeout eviction, VNC check for hung browsers. Anti-Patterns: Local Grid on laptop for CI; no `shm_size` (Chrome crashes); `close()` without `quit` (slot leak); static driver across parallel slots. Qs: How do slots vs threads vs nodes interact? What Grid metrics warn before queue stall?

21.37 Playwright in CI — Refined

Theory: Playwright excels in CI: headless Chromium/Firefox/WebKit, auto-waiting, tracing, sharding, HTML/JSON reporters built in. Install with `--with-deps` (system libs), shard via `--shard=1/3`, retry flaky, upload blob reports and merge to HTML. Docker or GitHub-hosted runners; `CI=true`; record video only on failure (storage/time). Enterprise Relevance: Playwright CI is simpler than Selenium Grid

(no Hub/nodes for most suites) — browsers launch per worker. Shard matrix + blob merge is the standard Deloitte pattern for 1000+ Playwright tests in <10min.

```
- run: npx playwright install --with-deps chromium
- run: npx playwright test --shard=${{ matrix.shard }}/3 --reporter=blob
- run: npx playwright merge-reports --reporter html ./blob-report
  if: always()
```

High-Stakes Scenario: CI installs all 3 browsers + deps every run (6min install, tests 4min). Triage: install chromium-only, cache browsers (~/.cache/ms-playwright keyed by version), Docker image with pre-installed browsers, shard 3x. Anti-Patterns: --with-deps without cache; headed in CI; trace: on always; full browsers for API-only PRs; no CI=true (retries/forbid-only skipped). Qs: Shard vs workers in Playwright CI? What does --with-deps install and how do you cache it?

21.38 REST Assured in CI — Refined

Theory: REST Assured API suites run in CI via Maven/Gradle Surefire/Failsafe, producing TestNG/JUnit XML for pipeline dashboards. Parameterize baseURI via environment variables, secrets for tokens, parallel execution, categories for smoke/regression. Fail pipeline on contract violations; publish Extent/Allure reports; archive logs. Combine with Newman/Postman or WireMock for isolated service virtualization when downstream is unstable. Enterprise Relevance: API suites are the fastest gate per defect found — 3min for 500 checks. Deloitte pattern: API smoke on every PR (blocking), full API + contract nightly, WireMock for third-party sandbox.

```
// baseURI from env, never hardcoded; token from secret
given().baseUrl(System.getenv("API_URL"))
    .header("Authorization", "Bearer " + System.getenv("API_TOKEN"))
    .when().get("/orders")
    .then().statusCode(200);
```

```
mvn -B verify -Dgroups=api-smoke      # PR gate
mvn -B verify -Dgroups=api-full       # nightly
```

High-Stakes Scenario: API suite hits shared staging, parallel runs collide (same order IDs) + downstream payment sandbox down → red. Triage: unique data per run (UUID), WireMock stubs for third-party, separate api-smoke (no external deps) vs api-full, contract tests versioned with provider. Anti-Patterns: Hardcoded staging URL/token; no categories (500 tests on every PR); hitting prod from CI; no WireMock (third-party flake blocks all). Qs: How do you isolate API tests from downstream instability? Smoke vs full API split?

21.39 Pipeline YAML / Jenkinsfile Code Snippets — Refined

Theory: Interview-ready snippets prove hands-on CI ownership: checkout, setup (Node/Java + cache), parallel UI/API testing, reporting. Keep pipelines DRY with reusable templates/anchors, shared libraries, timeouts, retry, allow_failure (advisory), caching, conditional stages. Always archive JUnit, screenshots, traces, coverage. Show declarative Jenkinsfile AND GitHub Actions matrix/sharding. Enterprise Relevance: SDET must write/modify pipelines, not just "run tests in Jenkins". Snippet fluency (timeout, retry, artifacts, matrix) is the difference between operator and owner in Deloitte interviews.

```
// Jenkinsfile — parallel UI/API + artifacts
pipeline {
  agent any
  options { timeout(time: 30, unit: 'MINUTES') }
  stages {
    stage('Test') {
      parallel {
        stage('API') { steps { sh 'mvn -B verify -Dgroups=api' } }
        stage('UI') { steps { sh 'npx playwright test --shard=1/2' } }
      }
    }
  }
  post { always { junit '**/TEST-*.xml'; archiveArtifacts 'playwright-report/**' } }
}
```

```
# Actions — matrix + sharding + cache
- uses: actions/setup-node@v4
  with: { node-version: 20, cache: npm }
- run: npx playwright test --shard=${{ matrix.shard }}/4 --reporter=blob
  timeout-minutes: 15
```

High-Stakes Scenario: Whiteboard "design our pipeline" — candidate recites tools, no YAML. Triage for interview: write triggers → stages → matrix/shard → artifacts → gates in 5min; narrate timeouts/retention/artifact decisions. Anti-Patterns: No timeouts (hung builds burn agents); no artifacts; copy-paste across 10 repos (use templates); `allow_failure: true` on critical (silent red). Qs: Write PR pipeline YAML from memory? What timeout/retention values and why?

21.40 CI/CD Interview Scenarios — Refined

Theory: Scenario questions test triage + judgment: flaky pipeline diagnosis, long runtime reduction, environment instability, secret leakage, broken main, quality gates. Answer with: triage logs → quarantine flakes → parallelization/sharding → test-pyramid rebalancing → canary rollbacks → Docker parity → SLAs. Emphasize shift-left, DORA metrics, coverage thresholds, blameless postmortems (Deloitte delivery culture). Enterprise Relevance: Every answer must end with metric + process change, not just fix. "Flaky? → quarantine + retry 2x + trace → root-cause ticket → unquarantine after 3 green runs" beats "rerun until green".

```
Flaky pipeline? → 1) quarantine label + trace, 2) waits/data/isolation fix, 3)
flake-rate dashboard, 4) gate new flakes.
45min → 10min? → cache (20→2) + shard (15→5) + smoke-first + cancel redundant.
Secret leaked? → rotate + purge + Gitleaks gate + storageState (once, not per test).
"Only run before release"? → escaped-defect cost data + DORA lead time + pilot (1
team, 4 weeks, measure).
```

High-Stakes Scenario: "CI is slowing us down, run automation only before release" (client leadership). Triage: cost-of-delay data (bugs found in PR: 2h fix; in release: 2d + hotfix), pilot proposal (smoke on PR for 1 squad, measure escape delta), risk framing (release-day surprises vs steady green). Anti-Patterns: Tool-only answers (no metrics); "add more agents" for every slowness; blaming devs ("they

break builds"); no postmortem/action items. Qs: Flaky pipeline triage order? How do you push back on "test less" with data?

21.41 Cloud-Native Scale: AWS ECS Fargate & Kubernetes Pods — Refined

Theory: Cloud-native testing targets ephemeral containers: ECS Fargate (serverless containers, no nodes) or EKS/K8s Jobs. Trigger via CodePipeline; run Playwright/Selenium as Kubernetes Jobs with parallelism; service discovery for test environments; ConfigMaps/Secrets for config; autoscale Grid nodes (HPA/KEDA on queue length). Validate health probes, rolling updates, ingress, HPA behavior, chaos resilience; teardown namespaces after execution to control cost. Enterprise Relevance: Static Grid VMs waste 80% cost idle; Jobs scale to zero. Deloitte pattern: e2e-shard Jobs (parallelism = shards), Grid nodes HPA on session-queue length, namespace per run with TTL auto-cleanup.

```
apiVersion: batch/v1
kind: Job
metadata: { name: e2e-shard-1 }
spec:
  parallelism: 4
  template:
    spec:
      containers:
        - name: pw
          image: registry/e2e:1.4.2 # immutable, same as app release
          envFrom: [{ configMapRef: { name: e2e-cfg } }, { secretRef: { name: e2e-
secrets } }]
          restartPolicy: Never
          ttlSecondsAfterFinished: 3600 # cost control
```

High-Stakes Scenario: Grid nodes idle weekend (\$2k/mo) yet Monday queue 30min. Triage: KEDA autoscale (0→10 nodes on queue depth), Jobs instead of Deployments (run-to-completion + TTL), spot instances for workers (70% cheaper, retry on preemption), namespace quotas to prevent runaway. Anti-Patterns: Permanent Grid deployment sized for peak (idle waste); no TTL (namespaces accumulate); prod cluster for load tests (noisy neighbor); missing probes (unhealthy nodes take sessions). Qs: Jobs vs Deployments for tests? How do you scale Grid nodes on demand? How do you bound cloud cost?

21.42 Execution Metrics & Logging Aggregation (ELK Stack & Grafana Loki) — Refined

Theory: Centralized logging validates distributed CI runs: ELK (Elasticsearch, Logstash, Kibana) or Loki/Grafana stacks. Stream app, Selenium, and pipeline logs with correlation IDs; structure JSON logs; dashboard pass/fail trends, latency p95, error rates, flaky hotspots. Alert via Grafana/PagerDuty on spike failures. SDET queries KQL/LogQL to triage defects and attaches evidence to Jira. Enterprise Relevance: Sharded/parallel runs scatter logs across 10 runners — centralization with correlation ID (run_id , shard , test) reassembles the story. Without it, triage means downloading 10 zips.

```
{job="e2e", run_id="412"} |= "AssertionError" | json | line_format "{{.test}}" failed
in {{.duration}}ms"
```

```
// structured test log – queryable, not grep-able text
{"ts":"2026-10-06T10:01:00Z","run_id":"412","shard":"2/4","test":"checkout","status":"fail","duration_
```

High-Stakes Scenario: Flaky hotspot invisible — same 5 tests fail 10% across shards, nobody connects them. Triage: flaky dashboard (group by test, not by run), LogQL top-failing tests last 7d, PagerDuty on new-flake (not every failure), Jira auto-file with log excerpt + trace link. Anti-Patterns: Plain-text logs (unqueryable); no correlation IDs; logs without test/run context; infinite retention (cost); PII/tokens in logs. Qs: What makes logs queryable vs grep-able? How do you alert on new flakes without spam?